

AI Product Development: Web Course Description

Catalog Year: 2027, Required Hours: 720, Credits: 24

Foundation Core (Required Hours: 360, Credits: 12)	Credits	Hours
Introduction to AI Product Development	1.00	30.00
Product Thinking and Planning	2.00	60.00
Software Systems and Architecture	2.00	60.00
Software Development with AI	3.00	90.00
AI Agents and Automation	4.00	120.00

Web Track (Required Hours: 360, Credits: 12)	Credits	Hours
Modern Front-End, AI-Accelerated	2.00	60.00
React & Component Architecture	3.00	90.00
Back-End & API Engineering	3.00	90.00
Cloud, DevOps & Web Security	2.00	60.00
Capstone & Career Launch	2.00	60.00

Foundation Core Courses (Required Hours: 360, Credits: 12)

Foundation Core (Required Hours: 360, Credits: 12)	Credits	Hours
Introduction to AI Product Development	1.00	30.00

The Introduction to AI Product Development course orients students to the world of AI-powered software development before any technical work begins. Students get a plain-language introduction to how AI systems work: what they are doing when they generate text, code, or decisions, and where they commonly fall short. The course also maps out what a modern technology company looks like: the roles on a product team and how those roles collaborate. The course closes with every student publishing a simple website built almost entirely with AI assistance.

Course Objectives:

- Explain in plain language how AI systems generate outputs and where they commonly fail.
- Identify the key roles on a software product team and describe how they collaborate.
- Describe how a technology organization is structured and where individual roles fit within it.
- Use AI tools to plan, build, and publish a simple website without prior technical experience.

Product Thinking and Planning	2.00	60.00
--------------------------------------	-------------	--------------

The Product Thinking and Planning course introduces the product development lifecycle from idea to launch: identifying a real user problem worth solving, gathering and prioritizing requirements, and translating a rough idea into a plan a team can act on. Students learn to write user stories and acceptance criteria and use a basic prioritization framework to decide what to build first and what to leave out.

Course Objectives:

- Describe the stages of a typical product development lifecycle from idea to launch.
- Identify a real user problem and articulate why it is worth solving.
- Write clear user stories and acceptance criteria.
- Gather, prioritize, and scope product requirements using a basic prioritization framework.
- Produce a one-page product brief that translates an idea into an actionable plan.

Software Systems and Architecture**2.00****60.00**

The Software Systems and Architecture course builds the conceptual foundation every student needs before writing code. Students explore how computers process and store information, learn core data structures, develop a working understanding of how the internet functions, and study the design patterns and data modeling principles that shape how real software is architected. Rather than analyzing an arbitrary app, the course project has students take a product idea of their own and produce the architectural plan for it: the data model, the API surface, the client-server structure, and how information would need to move through the system end to end. The goal is to build the habit of designing before building. Responsible data practices are introduced here too, with attention to what the proposed product would collect, why, and what students are accountable for as its future developers.

Course Objectives:

- Demonstrate understanding of computer architecture, memory, storage, and operating system concepts.
- Apply common data structures to organize and manage information in a program.
- Explain how the internet works, including HTTP, DNS, and client-server communication, and how APIs move data between services.
- Apply software design patterns and data modeling principles to plan how a system should be structured.
- Produce an architectural plan for a product idea, including its data model, API surface, and system boundaries.
- Identify responsible data collection practices and describe the privacy implications of design decisions.

Software Development with AI**3.00****90.00**

The Software Development with AI course is where students go from a plan to a working system. Students set up a professional development environment, write code in a general-purpose programming language, and use AI coding assistants and prompt engineering as core tools. Throughout the course, students critically evaluate the code AI produces, write tests to verify that AI-generated code actually does what it claims before it ships, and practice the judgment to know when to trust AI output and when to rewrite it. Ethical use is embedded in all projects through the principle that the developer is responsible for everything they ship, regardless of how it was written.

Course Objectives:

- Set up and navigate a professional software development environment including version control.
- Write and organize code using a general-purpose programming language.
- Use AI coding assistants and prompt engineering to accelerate development without sacrificing understanding.
- Critically evaluate AI-generated code for correctness, security, and appropriateness before shipping.
- Write tests to verify that AI-generated code behaves as intended before it ships.
- Build and deploy a scoped application that solves a defined, familiar problem using AI assistance.
- Design, build, and present a self-directed final project that turns a product idea into a working system.
- Articulate the developer's ethical responsibility for AI-assisted output.

AI Agents and Automation

4.00

120.00

The AI Agents and Automation course teaches students to design, build, and deploy agentic AI systems and automated workflows. Starting with no-code and low-code platforms, students progressively work with direct API integrations and agent frameworks to build systems that take real actions on behalf of users. The course covers how agents perceive inputs, reason through tasks, and produce outputs; how to design and sequence multi-step agentic workflows; and how to evaluate and debug agent behavior in production. Ethical considerations are central throughout: students examine the risks of autonomous systems, determine when human oversight is required, build appropriate guardrails, and reason through what accountability looks like when an agent makes a mistake that affects a real person.

Course Objectives:

- Build automated workflows using no-code and low-code platforms.
- Connect applications and services using APIs, webhooks, and event triggers.
- Explain how AI agents perceive inputs, reason through tasks, and produce outputs.
- Design and implement a multi-step agentic workflow using an agent framework.
- Evaluate and debug agent behavior using logging, testing, and prompt refinement.
- Identify failure modes of autonomous systems and implement appropriate guardrails.
- Apply human-in-the-loop design patterns in contexts where oversight is required.
- Articulate an accountability framework for agentic systems that affect real users.

Web Track Courses (Required Hours: 360, Credits: 12)

Web Track (Required Hours: 360, Credits: 12)	Credits	Hours
--	---------	-------

Modern Front-End, AI-Accelerated

2.00

60.00

The Modern Front-End, AI-Accelerated course teaches students to build the browser platform before adopting a framework. Building on the shared core's programming fundamentals, HTTP, Git workflow, and AI-assisted development practices, students create semantic, responsive, and accessible interfaces with HTML, CSS, JavaScript, and TypeScript, and connect those interfaces to live web services. Students use browser developer tools to debug, audit, and improve real application behavior, with checkpoints requiring them to critique, correct, test, and justify AI-generated code.

Course Objectives:

- Build semantic, accessible HTML and implement responsive layouts using the box model, Flexbox, Grid, media and container queries, and reusable design tokens.
- Write modular, typed client-side code in JavaScript and TypeScript using functions, objects, arrays, modules, events, promises, and async/await.
- Connect pages to live data from web services and handle loading, empty, success, and error states without losing a user's work.
- Build accessible, validated forms while distinguishing client-side convenience validation from server-side security controls.
- Debug client-side applications using browser developer tools and write unit tests for deterministic logic.
- Evaluate AI-assisted code against product requirements, tests, accessibility, security, and licensing expectations.

React & Component Architecture**3.00****90.00**

The React & Component Architecture course has students use React and TypeScript to transform product requirements into multi-route web applications. Students design reusable component systems, manage local and shared state, build accessible forms, implement client-side navigation and authentication workflows, and connect applications to live APIs and external services. They test applications from the user's perspective, measure and improve performance, safely refactor existing code, and apply core AI competencies by building an AI-powered interface with streaming responses and meaningful user controls.

Course Objectives:

- Design maintainable component architectures by breaking product requirements into organized, reusable React components.
- Manage application state and behavior using React hooks, predictable data flow, Context, and appropriate state-management patterns.
- Build complete multi-route applications with client-side navigation, authentication workflows, protected routes, and accessible forms.
- Integrate live APIs and external services while handling loading, empty, success, optimistic updates, validation, and error states.
- Write user-focused tests, measure application performance, and implement evidence-based improvements.
- Review code, communicate architectural decisions, and safely refactor duplicated or difficult-to-maintain code.
- Develop responsible AI-powered features that support streaming responses, cancellation, error recovery, and meaningful user control.

Back-End & API Engineering**3.00****90.00**

The Back-End & API Engineering course has students design and implement secure, production-oriented server applications using Node.js, TypeScript, Express, and PostgreSQL. Students model relational data, develop documented REST APIs, validate untrusted input, implement authentication and authorization, and diagnose failures through structured logging and automated tests. Students then compare that relational architecture with Firebase and Firestore, and securely integrate an AI service behind a protected server boundary with credential protection, validation, rate and cost controls, and user-safe fallback behavior.

Course Objectives:

- Explain the Node.js runtime environment, including asynchronous I/O, package management, and environment-based configuration.
- Design, implement, and document RESTful APIs in TypeScript with typed routes, middleware, status codes, pagination, and an OpenAPI contract.
- Model relational domains in PostgreSQL and write SQL for CRUD operations, joins, aggregates, and transactions; apply keys, constraints, normalization, and migrations.
- Implement layered application security, including input validation, secure credential storage, session or token authentication, and least-privilege authorization.
- Write unit and integration tests covering success, validation, authentication, authorization, and failure paths, and implement structured logging and health checks.
- Build a Firestore feature with security rules, justify relational vs. document-database vs. backend-as-a-service tradeoffs, and integrate an AI service behind a secured boundary with rate, cost, and fallback controls.

Cloud, DevOps & Web Security**2.00****60.00**

The Cloud, DevOps & Web Security course has students deploy and operate a full-stack application in a repeatable, production-like cloud environment. They apply Linux and networking fundamentals, containerize multi-service applications with Docker, configure a focused AWS deployment architecture, build automated CI/CD pipelines with GitHub Actions, and implement monitoring and recovery practices. Security, accessibility, performance, search discoverability, AI-service reliability, and cost awareness are treated as measurable release requirements rather than afterthoughts.

Course Objectives:

- Diagnose application delivery problems using Linux command-line, process, file-permission, environment, port, DNS, and HTTP/TLS concepts.
- Construct production-oriented container images and deploy a containerized web application and database with Docker and Docker Compose across development, test, and production configurations.
- Configure secure AWS environments applying identity and access management, least-privilege principles, domains, DNS, HTTPS, and secrets management.
- Build an automated CI/CD pipeline with GitHub Actions that installs, lints, tests, scans, builds, and deploys the application, with health checks, logging, and a documented rollback procedure.
- Identify and remediate representative OWASP Top 10 security risks, including access control, injection, authentication, and supply-chain failures.
- Evaluate release readiness against a production gate covering accessibility, Core Web Vitals, caching, SEO, error handling, and AI-integration monitoring.

Capstone & Career Launch**2.00****60.00**

The Capstone & Career Launch course has teams of two synthesize the competencies of the full program by delivering and defending a deployed full-stack product while maintaining documented evidence of individual contribution. Students validate the problem, refine requirements, define acceptance criteria, and determine how AI contributes to the product's value, managing work through issues, pull requests, and architecture decisions. Career preparation converts the capstone into an employer-ready portfolio and develops the code-review, debugging, and interviewing skills expected of entry-level developers.

Course Objectives:

- Convert an ambiguous stakeholder problem into a bounded product brief with a prioritized backlog, acceptance criteria, data model, and delivery plan.
- Justify and document proportionate architecture decisions and execute them through a professional team workflow of small issues, branches, and code review.
- Construct an accessible and responsive React and TypeScript client integrated with a documented, secure, data-backed API and a bounded AI capability, delivered as complete vertical slices.
- Validate the product through unit, component, integration, and end-to-end tests, and deploy through a CI/CD pipeline with verified security, accessibility, and recovery procedures.
- Prioritize defects and technical debt by user impact, risk, effort, and release timing, and defend individual code contributions, including AI-assisted work.
- Present an employer-ready professional portfolio and demonstrate entry-level interview competencies through live coding, code review, and behavioral examples.